

⑤ 3 methods of finding the size of a file

`ls -lh file`
 ↓
 list command
 ↳ long, human-readable format

`du -h file`
 ↓
 disk usage

`stat -c %s file`
 ↓
 metadata of the file (detailed status)
 ↳ custom → size in bytes

`wc -c file`
 ↓
 word count
 ↳ count bytes

⑥ UNIX shell → check if $\exists$ file and current user has any permissions to

```
if [ -e "$FILE" ] && [ -r "$FILE" -o -w "$FILE" -o -x "$FILE" ]
then
  echo "Exists and has permissions"
else
  echo "Does not exist or no permissions"
fi
```

↑ readable OR writable OR executable

modern Bash:

```
if [[ -e "$FILE" ]] && [[ -r "$FILE" || -w "$FILE" || -x "$FILE" ]]
then
  echo "exists w permissions"
else
  echo "..."
fi
```

⑤ 3 methods of finding the size of a file.

`ls -lh file`
↓
list command
long → human-readable format

`du -h file`
↓
disk usage

`stat -c %s file`
↓
metadata of the file (detailed status)
custom → size in bytes

`wc -c file`
↓
word count
count bytes

⑥ UNIX shell → check if file and current user has any permissions to it.

```
#!/bin/bash
FILE="file.txt"
if [ -e "$FILE" ] && [ -r "$FILE" -o -w "$FILE" -o -x "$FILE" ]
then
    echo "Exists and has permissions"
else
    echo "Does not exist or no permissions"
fi
```

modern Bash:

```
if [[ -e "$FILE" ]] && [[ -r "$FILE" || -w "$FILE" || -x "$FILE" ]]
then
    echo "exists w permissions"
else
    echo "..."
fi
```

Process hierarchy, including parent.

```
for (i=0; i<3; i++) {  
    if (fork() > 0) {  
        wait(0);  
        wait(0);  
        wait(0); exit(0);  
    }  
}
```

~~fork~~
→ C1 → wait(0) → exit(0)
→ C2 → wait(0) →
→ C3

i=0

P → enter if() → wait(0) → stuck waiting for C1
→ C1 → skip if() (i=0) ✓

i=1

P
→ C1 → enter if() → wait(0) → stuck waiting for C2.
(i=0)
→ C2 → skip if() (i=1)

i=2

P
→ C1
(i=0)
→ C2 → enter if() → wait(0) → stuck ...
(i=1)
→ C3 → skip if() (i=2)

⑧ Printed? → full path required ⇒ exec fails! ⇒ returns -1

exec("expr", "expr", "1", "+", "1", NULL);

execp("echo", "echo", "3", NULL);

printf("4\n");

⇒ succeeds

→ automatically searches for path ⇒ does not return

⇒ output: 3

⑨ Why the file descriptor ~~opened with~~ returned by popen must be closed with pclose instead of fclose?

popen() → opens a new process that needs to be closed with pclose(); pclose() closes the file descriptor, waits for the associated process to finish and returns the termination status, while fclose() would only close the file stream.

pclose() → close file pointer, invoke waitpid(), return exit code

fclose() → close stream, but child process is zombie (no cleanup)

✓ (10) How many FFDs can a process open for reading if only... for writing?

✓ (11) execv instead of exec?

execv() → when I need to pass an array of args.

→ when the nr. of arguments is dynamic / known at runtime

✓ (12) critical section

✓ (13) 3 function calls that ensure mutual exclusion.

pthread_mutex_lock(&mutex)

sem_wait(&sem)

pthread_rwlock_wrlock(&rlock)

✓ (14) pthread_mutex_lock ↔ sem_post?

⇒ chaos because sem_post increases the semaphore, allowing any thread to enter ⇒ race conditions

✓ (15) if executed by several simultaneous threads?

pthread_mutex_lock(&first);
----- lock(second);

----- unlock(second);

----- unlock(first);

Thread A: - holds m[0]
- waits for m[1]

Thread B: - holds m[1]
- waits for m[0]

⇒ DEADLOCK. (locking order depends on parity)

(if one has m[0] and the other m[1], they will attempt to lock the same ~~mutex~~ two mutexes in rev. order).

✓ (16) $T, N_1, N_2, \neq, > 0$.

pthread_barrier_t b1, b2; → declare 2 barriers

void* f1(void* a) {
pthread_barrier_wait(&b1);
return NULL;
}

void* f2(void* a) {
----- (&b2);
}

int main() {

int i;

pthread_t t[T][2]; ← T rows, 2 threads/row $T=4, N_1=2, N_2=1$

pthread_barrier_init(&b1, NULL, N1); → N1 = barrier limit for b1

----- (&b2, NULL, N2); → N2 = barrier limit for b2

for(i=0; i<T; i++) { ← repeat T times

----- create(&t[i][0], NULL, f1, NULL); ← create one f1 thread

----- ([1], ---, f2, ---); ← create one f2 thread

----- join ← wait for all threads

----- destroy

$T \% N_1 = 0$

$T \% N_2 = 0 \Rightarrow N_1=2, N_2=1$.

①7 prevent deadlocks ✓

①8 state transition sem-wait, under what conditions?

sem-wait() $\rightarrow$ if value $> 0 \Rightarrow$ decrement counter (value)
 $\Rightarrow$ remain in running state

$\rightarrow$ if value $= 0 \Rightarrow$ decrement $\Rightarrow -1 < 0 \Rightarrow$

$\Rightarrow$ waiting / blocked state until another process / thread increments using sem-post() $\Rightarrow$ ready

✓ Semaphore = an integer whose value is never allowed to fall below 0.
 $\hookrightarrow$ allows 2 operations -

- ① increment the semaphore by one (sem-post())
- ② decrement (sem-wait())

$\hookrightarrow$ if $= 0 \Rightarrow$ a sem-wait() op. will block until value > 0 .

①9 B = block size ✓
A = addr. size

How many data blocks are addressed by the double indirect addr. of an i-mode?

$$(B/A)^2$$

②0 when to load in memory the pages of a process that is just starting?

Demand paging: load only when needed.

$\hookrightarrow$ initially load only the pages needed to start execution;
other pages are loaded when the process first accesses them

$\hookrightarrow$ optimisation: - spatial: nearby pages } locality principle
(preemptive) - temporal: recently-used } optimisation

$\downarrow$
(the OS may prefetch nearby pages based on LOCALITY to reduce future page faults).